

2026-2 / IIC2173 - E0 | EnergyShark

Fecha de entrega: 30/ago/2026

Introducción al proyecto del curso

A nivel general, el proyecto del curso busca dar a los alumnos una experiencia práctica del desarrollo de un sistema de software compuesto de diferentes componentes que interactúan de formas complejas, su modelación y documentación. Dicho sistema será desarrollado en grupos y la idea es que este desarrollo sea lo más parecido a un sistema real posible. Para partir con esto, la entrega 0 es de carácter individual y en futuras entregas se trabajará en grupo. Así, el proyecto abarca temáticas como protocolos, integraciones, seguridad, deployment, entre otros. El proyecto deberá **siempre** correr en un entorno Cloud para obtener los conocimientos prácticos del desarrollo y deployment de software en la nube.

Enunciado

LegitBusiness, imperio corporativo a nivel global ha entrado en el negocio del movimiento de bolsas de energía con una plataforma llamada EnergyShark. Esta plataforma sirve para manejar el flujo de energía eléctrica entre ciudades

Debido a las diferencias en generación de energía, cada una de estas ciudades genera un balance interno de energía, positivo o negativo. La idea es que posteriormente puedan movilizar energía y hacer diversas tareas con este balance.

Como primer avance encargado por su Product Owner (el equipo docente), se van a familiarizar con el ecosistema básico del proyecto para posteriormente manejar todas las fuentes de energía del país

Para empezar, van a escuchar eventos básicos del exchange de la central con un *observer*, así como registrarlos para consultarlos y confirmar su recepción.

El evento clave que van a registrar ahora es el de demanda de energía en su ciudad. Este registro sencillo permite saber cuanta demanda tienen en una ciudad dada.

Estos eventos tienen el siguiente schema JSON

```
{
  "idpk": "UUIDv4",
  "type": "demand-set",
  "packageBody": {
    "demands": [
      {
        "city": "New New york",
        "demand": 1013.123,
        "unit": "GW"
      },
      //más ciudades
      {
        "city": "Los Santos",
```

```

        "demand": 10223,
        "unit": "GW"
    },
    ],
    "validUntil": "2026-12-12T00:00:00Z",
    "metaContent": "...",
    "constraints": {}, //vacío por ahora
}
}

```

Donde *idpk* es un identificador único para el evento (no repetible) y *type* es el tipo de evento, que por ahora solo es "demand-set". Los campos que refieren a tiempo se entregan como fechas ISO8601 en UTC

En este momento, el contenido de los eventos no es importante, lo importante es que puedan recuperarlo y guardarlo en base de datos para consulta posterior. Deben guardarlo con el timestamp **receivedAt**

Sobre el sistema de eventos

Cada uno de ustedes tendrá credenciales para su *observer*, para que así puedan conectarse al exchange. Cada observer se conecta a una cola llamada *observer.X.q* donde X representa un número de usuario que se les va a asignar para la tarea junto con sus credenciales. Esta información les llegará mediante un mensaje en Canvas.

Esta cola, la cual va guardando los mensajes que llegan, recibe del exchange *fulfillment.x* el cual está alimentado por la central de eventos. Utilizando el cliente de RabbitMQ en su aplicación podrán consumir los mensajes de esta cola. Cada consumo sacará el elemento de la cola y podrán leerlo y procesarlo.

El broker se encuentra actualmente ubicado en

```
broker.iic2173.org:5671
```

Y será activado el lunes 17 de agosto a las 00:00 La información de los mensajes viene en un string JSON que deben parsear para revisar que llegue igual que el schema

Especificaciones técnicas

Para esta tarea personal, "Entrega 0", la idea es que todos aprendan la base de una aplicación montada en la web que sea capaz de integrarse a un software externo, por lo que deberán crear y configurar un servicio web que implemente un pequeño servicio que funcione como API.

Pueden desarrollar su solución con el framework que deseen de esta lista:

Servicio web:

Ruby

- Rails (Version API)

Python

- FastAPI
- Django

Javascript

- Koa
- Express
- **NestJS**

Lenguajes para el servicio de conexión a broker:

- Ruby
- Python
- Javascript/typescript
- Go
- C#
- C/C++
- etc..

Queda prohibido y no se revisarán aplicaciones hechas en PHP.

Cada servidor tendrá que tener un dominio asignado. Los dominios ME pueden ser conseguidos fácilmente y de forma gratuita en Namecheap utilizando el Github Student Pack. También pueden usar otro proveedor de dominio si estiman conveniente.

Para SSL, podrán usar Let's Encrypt con certbot, un servicio gratuito de implementación de HTTPS.

Finalmente, solo pueden usar los siguientes proveedores en la nube, y sus servicios IaaS:

- AWS

Los ayudantes podrán responder sus dudas en AWS. Además, las ayudantías se referirán a AWS. Las siguientes herramientas / plataformas están prohibidas y no se corregirá nada que ocupe estos servicios:

- Heroku
- Ciertos servicios de AWS como:
 - LightSail
 - Elastic Beanstalk
 - Amplify
 - Cognito (para esta entrega)
- Netlify

- Firebase (excepto para notificaciones móviles)

En general, el servicio principal y el connector deben ser desplegados y administrados sobre EC2. No podrán utilizar plataformas que abstraigan su deployment o ejecución. Se permite usar RDS exclusivamente como servicio de persistencia.

En caso de que no tengan tantos conocimientos de desarrollo web recomendamos que usen stacks tecnológicos que ya conozcan o que sean usados en las ayudantías. Por otro lado, si quieren indagar en nuevos stacks tecnológicos pueden usar el proyecto del curso para explorar nuevas tecnologías. Recuerden que luego, en futuras entregas, el proyecto se vuelve grupal, así que vean con sus compañeros qué es lo que más les acomoda para que puedan reutilizar la mayor cantidad de código posible. Recuerden que no hay mejores tecnologías sino que algunas son más adecuadas para algunos contextos o tareas.

Puntaje

Esta entrega consiste en dos partes, la parte mínima (que todos deben lograr) que vale 75% de la nota final y una parte variable que también vale 25%. Sobre la parte variable, tendrán 2 opciones para trabajar, de las que deberán escoger 1. Cada una de las que escojan para evaluar vale 25% de la nota final, y realizar la segunda parte puede dar hasta 5 décimas.

Los requisitos marcados como **Esencial** son obligatorios para que su tarea sea revisada, esto pues o son fundamentales para el aprendizaje, o son necesarios para las siguientes entregas del proyecto. De no cumplir con estos, su tarea no será revisada y será calificada con nota 1.

Requisitos funcionales (10p)

Consideren que no se les pide crear una interfaz para su aplicación (sin HTML), sino que debe ser una API que entregue la información.

RF1: (3p) Esencial

Debe poder ofrecer en una API la lista del historial de demanda de electricidad. Deben incluirse al menos todos los campos presentes en el historial

RF2: (1p)

Debe ofrecer un endpoint para mostrar el detalle de cada registro de demanda de electricidad recibido desde el broker. La URI debe ser: `{url}/history/{:id}`. Acá, el campo ID sera el generado por ustedes, el cual debe ser mostrado en el endpoint del RF4.

RF3: (2p) Esencial

La lista de demandas de electricidad debe estar paginada por default para que muestre cada 25 registros y poder cambiar de pagina cambiando un queryParam. Es decir: `{url}/history?page=2&limit=25`. Queda a criterio de ustedes si permiten traer más valores mediante otro número del limit en queryParams.

RF4: (4p) Esencial

El endpoint `{url}/history` debe permitir filtrar las demandas por cada una de sus propiedades, especialmente los relacionados con tiempo. Por ejemplo: `{url}/history?receivedAt=2025-08-08`

Nota: es importante que sea paginada pues el emisor de eventos enviará miles de paquetes durante el desarrollo y corrección de sus proyectos, por lo que si hacen una consulta a la base de datos para que traiga todos los eventos recibidos es probable que se caigan sus entregas.

Requisitos no funcionales (20p)

RNF1: (5p) Esencial

Debe poder conectarse al broker RabbitMQ mediante el protocolo AMQP usando un servicio que corra de forma constante e independiente de la aplicación web en un container aparte, llamado *connector*. Este servicio debe alimentar a su servicio principal mediante llamadas HTTP POST. Los eventos recibidos deben ser persistidos con su sistema para que estos puedan ser mostrados bajo demanda. Además, su entrega debe resistir caídas de servicio del broker. Si se pierde temporalmente la conexión con RabbitMQ, *connector* no debe terminar permanentemente y debe intentar reconectarse sin intervención manual. La indisponibilidad del broker no debe impedir que *master* continúe atendiendo consultas sobre eventos previamente almacenados.

NO se corregirán entregas caídas

RNF2: (4p) Esencial

Su despliegue debe estar containerizado. Debe existir un servicio dentro de un container llamado *master* capaz de recibir los datos de *connector*, en la misma *network* docker de *connector*

RNF3: (3p)

Debe haber un proxy inverso apuntando a su aplicación web (como Nginx o Traefik). Esta instancia debe ir configurada directamente sobre su máquina EC2, no dentro de un contenedor Docker.

RNF4: (2p)

El servidor debe tener un nombre de dominio registrado bajo un tld publico (tech, me, tk, ml, ga, com, cl, etc)

RNF5: (2p) Esencial

El servidor debe estar corriendo en una máquina EC2 con free tier.

RNF6: (2p)

Debe haber una base de datos Postgres o Mongo externa asociada a la aplicación para guardar eventos y consultarlos. Esto puede ser en un container dentro de la máquina EC2 o en una instancia RDS.

RNF7: (2p) Esencial

Todos los containers deben tener una instrucción HEALTHCHECK que use curl o file checking para verificar que su servicio esté operativo y que pueda operar. No es necesario que connector exponga una API HTTP exclusivamente para este propósito.

Docker-Compose (15p)

Componer servicios es esencial para obtener entornos de prueba confiables, especialmente en las máquinas de los desarrolladores. Además, esta herramienta será necesaria durante el resto del desarrollo del proyecto para orquestar sus contenedores y servicios.

RNF1: (5p)

Lanzar *master* desde docker compose

RNF2: (5p)

Integrar su DB desde docker compose

RNF3: (5p)

Lanzar *connector* desde docker compose y conectarlo al contenedor de la app web

Variable

Deben elegir al menos uno de los dos grupos de requisitos siguientes.

HTTPS (25%) (15p)

La seguridad es esencial para sus usuarios, por ello los datos que viajan en su aplicación web deben encriptarse con los protocolos correspondientes.

RNF1: (7p)

El dominio debe estar asegurado por SSL con Let's Encrypt.

RNF2: (3p)

Debe poder redireccionar HTTP a HTTPS.

RNF3: (5p)

Se debe ejecutar el chequeo de expiración del certificado SSL de forma automática 2 veces al día (solo se actualiza realmente si está llegando a la fecha de expiración).

Algunos software de terceros requieren que sus aplicaciones funcionen con HTTPS, por lo que sin esta capa de seguridad no se puede integrar a dichos servicios. En futuras entregas será necesario tener implementado HTTPS por lo antes mencionado, por lo que si lo logran mejor!.

Balanceo de Carga con Nginx (25%) (15p)

Para escalar el servicio de forma horizontal podemos replicar el contenedor de la aplicación web y utilizar NGINX como balanceador de carga, así podemos crear tantos servidores como nuestro servicio necesite. Esto es altamente necesario para aplicaciones que en ciertos momentos tendrán una carga importante y en otros no.

RF1: (5p)

Debe replicar *master* en al menos 2 instancias container para que corran en paralelo.

RF2: (10p)

Cada instancia de master debe ser individualmente alcanzable desde el Nginx instalado en el host. La estrategia utilizada para exponer dichas instancias queda a criterio del estudiante.

Recomendaciones

Para esta entrega basta con que se pueda llevar a cabo las funcionalidades solicitadas manteniendo el estandar de una API. Fíjense en la proporcion entre RNF y RF. **No será necesario hacer una interfaz visual.**

Roadmap sugerido

Para simplificar esta primera entrega, le sugerimos seguir los siguientes pasos:

1. Ejecute pruebas de concepto para conectarse al cliente RabbitMQ.
2. Levante un servidor web HTTP que pueda leer los eventos recibidos.
3. Ponga su servicio en un container Docker.
4. Levante una máquina en AWS EC2 y abra los puertos.
5. Instale docker en la máquina.
6. Copie su aplicacion y construya el container.
7. Configure DNS para su servicio
8. Corra su servicio y termine su configuración.

Todo lo que es Nginx es mejor configurarlo directamente en la instancia EC2.

Recuerden que

- La entrega debe estar corriendo en el EC2 en cloud, **no se corregirán entregas locales**, por lo que avancen incrementalmente en el cloud.
- Funcionalidad que no esta en producción no está realmente terminada, no subestimen la dificultad del deployment, pues es mucho más manual que en otros cursos como ingeniería de software o desarrollo web.
- Si intentan avanzar con un servidor MCP sobre AWS, van a obtener sugerencias muy radicales de deployment usando herramientas avanzadas

Entrega

Se les proporcionará un repositorio de Github Classroom donde pueden subir su código e ir registrando sus commits.

Deben subir el código de su solución junto al archivo de configuración de Nginx (o Traefik u otro) en el repositorio que se les asignará vía github classroom.

También, se les publicará un buzón en canvas donde deben entregar el archivo `.pem` asociado al servidor EC2, además de la información para conectarse al servidor y sus objetivos logrados. Esto con el fin de tener los respectivos accesos y poder realizar una buena corrección.

Así, para poder facilitar la corrección deben generar un `README.md` que señale:

Consideraciones generales

- Nombre del dominio
- Método de acceso al servidor con archivo `.pem` y ssh (no publicar estas credenciales en el repositorio).
- Puntos logrados o no logrados y comentarios si son necesarios para cada aspecto a evaluar en la Parte mínima y en la Parte variable.
- Pueden sobrescribir este README sin problemas o cambiarle el nombre

Causas de rechazo de entrega (Nota 1 directo):

- **ESTÁ ABSOLUTAMENTE PROHIBIDO SUBIR SU ARCHIVO `.PEM` A SU REPOSITORIO DE GITHUB. Si hacen esto se les calificará con nota 1.** Para esto se les habilitará un buzón de canvas para que nos lo compartan. **No subir este archivo a canvas implica que su entrega no se corregirá**
- Entregas que no estan healthy cuando se vayan a revisar
- Dominios que no apunten a la instancia

Esta entrega es estrictamente individual y será revisada para casos de copia.

Atraso

Para esta entrega se les descontará 0.5 puntos en la nota máxima por horas Fibonacci con $F1 = 6$ y $F2 = 6$.

Se considerará como atraso cualquier modificación en features o implementación que tenga que ver solo con lo que se pide en esta entrega.

Fibonacci	Hora	Nota máxima
6	0:01 - 5:59	6.5
6	6:00 - 11:59	6
12	12:00 - 23:59	5

Fibonacci	Hora	Nota máxima
18	24:00 - 41:59	4.5
30	42:00 - 71:59	4
...	...	...
	72:00 en adelante	1

Respecto al uso de AI

El flujo de esta entrega es liberal pero debe documentarse

- Pueden usar tab autocomplete
- Si hacen consultas a ChatGPT/Claude deben registrar cada prompt en el directorio `ai-docs/prompts` con su respuesta
- Si utilizan cursor o algun AI-assisted IDE, deben registrar cada interaccion con este agente. Esto es, deben incluir todo el historial de chat bidireccional relacionado con esta entrega
-

Enlaces relevantes

- [RabbitMQ - Tutorial JS](#)
- JSON strings parsing
 - [Python](#)
 - [JS](#)
 - [Ruby](#)
- [Github Student Pack](#)
- [Página AWS](#)
- [Docker Engine - Instalación](#)
- [Conectarse con SSH](#)
- [Nginx - Reverse Proxy](#)
- [Nginx - Instalar en Ubuntu](#)
- [Certbot - SSL Let's Encrypt](#)

Ayudantías Útiles

- Ayudantía 1 - Cloud 1: AWS, Linux y EC2
- Cápsula 1 - Cloud 2: Docker, Docker-Compose y alternativas
- Ayudantía 2 - Cloud 3: Deployment básico

Apoyo

Pueden usar el Slack del curso en el canal  para dudas más rápidas.